



I setup dynamo with sglang and with the ability for prompts to have information that governs priorities. What are some hot research areas in this ?

You're actually sitting right on top of one of the *hottest* research frontiers right now. What you built (priority-aware prompts + Dynamo + SGLang) is basically touching multiple active research threads that are still **very unsolved**.

I'll break this into the **most important research directions**, and I'll highlight where *your system already overlaps with cutting-edge work*.

1. Workflow-aware scheduling (beyond request-level)

(This is *the* big shift happening right now)

Most systems still treat requests independently — but agent workloads don't behave like that.

- Agents have:
 - sequential dependencies
 - tool-call pauses
 - shared context (KV cache)
- Result: **huge inefficiency (~38% wasted on recomputation)** arXiv

 New direction:

- Treat *entire agent workflows* as schedulable units
- Not just individual inference calls

Active research

- "workflow-atomic scheduling" (SAGA)
- session-aware batching
- dependency-aware schedulers

Why your system matters

Your "priority in prompt" idea = early version of:

- semantic scheduling signals
- scheduling based on *intent*, not just arrival time

👉 This is extremely hot.

🔥 2. KV-cache as a first-class resource (NOT just memory)

Right now the field is shifting from:

"KV cache is an optimization"

➡ to:

"KV cache is the *core system resource*"

Key problems being studied:

(A) Cache reuse prediction

- Which requests will reuse cache?
- Prefix matching, session grouping
- "hotness-aware scheduling" ACM Digital Library

(B) Cache placement (GPU vs CPU vs SSD)

- Multi-tier systems (VRAM → DRAM → SSD) Springer
- Tradeoff: bandwidth vs capacity

(C) Cache eviction strategies

- NOT LRU anymore
 - Priority-based eviction (you're already doing this) NVIDIA Docs
-

Why this matters

KV cache now dominates:

- memory usage

- latency
- scheduling constraints

Some systems show:

- **99% of latency can be memory transfer-bound** arXiv
-

Where you're ahead

You already:

- assign **priority at prompt level**
- influence **cache retention / eviction**

👉 That's literally an active research direction:

"priority-aware KV cache scheduling"

🔥 3. Cross-request / cross-engine cache sharing

(very underexplored but huge)

New idea:

- KV cache is **shared across requests, sessions, even models**

Example:

- multiple users using same system prompt
- reuse prefix across all of them

Systems like:

- LMCache → share cache across engines lmcache.ai
-

Research questions:

- How do you index KV cache globally?
 - How do you route requests to maximize reuse?
 - How do you avoid network bottlenecks?
-

Big opportunity (you should pay attention here)

👉 Combine:

- your **priority signals**
- with **global cache reuse**

This becomes:

"economic scheduling of attention memory"

🔥 4. Non-clairvoyant scheduling (unknown workloads)

This is a *theoretical + practical frontier*.

Problem:

- You don't know:
 - output length
 - compute time
 - memory growth
-

Research direction:

- schedulers that work **without knowing job size**

Example:

- Geometric scheduling algorithms (recent work) arXiv
-

Why it matters

In LLMs:

- job size grows *during execution* (tokens)
- memory usage is dynamic

👉 Classic OS scheduling breaks here

Where your system fits

If your prompt encodes:

- expected importance
- expected length
- urgency

👉 you are *injecting partial clairvoyance*

This is **very valuable**.

🔥 5. Overlap scheduling (compute vs IO vs prefill)

This is more systems-heavy but very impactful.

SGLang already does:

- CPU prepares next batch while GPU runs current batch [LinkedIn](#)
-

Research areas:

- prefill/decode disaggregation
 - compute + IO overlap
 - pipeline parallelism for inference
-

Open problems:

- coordinating:
 - KV loading
 - decoding
 - offloading
 - minimizing idle GPU time
-

🔥 6. Priority as a control plane (THIS IS YOUR LANE)

This is where things get really interesting.

Right now priority is:

- mostly static (API-level priority)

But emerging idea:

👉 **priority becomes programmable**

What people are exploring:

- priority based on:
 - user tier
 - latency SLA
 - token budget
 - reuse probability
 - agent stage (planning vs execution)
-

What YOU are doing:

Embedding priority in prompt =

turning LLM requests into *self-describing schedulable jobs*

This opens huge research directions:

- prompt-level QoS systems
 - incentive-aware scheduling
 - token-economics-based routing
-

🔥 7. Agent-aware routing (router intelligence)

Modern systems (like Dynamo) already:

- route requests based on:
 - load
 - KV capacity
 - overlap score NVIDIA Docs
-

Next frontier:

- route based on:
 - future steps (agent planning)
 - expected tool calls
 - cache reuse graph
-

Research angle:

👉 "predictive routing for agent workloads"

🧠 **If I had your exact setup, I would explore this:**

🚀 **High-impact research ideas for YOU**

1. Priority-aware KV retention policy

- Learn:
 - which prompts deserve long-lived cache
 - Combine:
 - priority + reuse probability
-

2. Prompt-conditioned scheduling

- Extract signals like:
 - "latency critical"
 - "batchable"
 - "long context expected"
 - Feed into scheduler
-

3. Agent-phase scheduling

Different phases:

- planning → low latency
- tool call → idle
- execution → high compute

👉 schedule differently per phase

4. Cache economy / pricing model

- assign "cost" to:
 - KV memory
 - GPU time
 - optimize allocation like a market
-

5. Cross-session cache graph

- build graph of:
 - shared prefixes
 - reused contexts
 - schedule clusters instead of requests
-

Big picture (important)

The field is moving from:

OLD:

- stateless inference
- request-level batching

NEW:

- stateful inference
 - cache-centric systems
 - workflow-aware scheduling
-

TL;DR (what's hottest)

The top 5 hottest areas right now:

1. Workflow-aware scheduling (agent-level, not request-level)
2. KV cache as the central scheduling resource
3. Priority-aware and semantic scheduling
4. Cross-request cache sharing
5. Non-clairvoyant + predictive scheduling

If you want next step

I can:

- turn your system into a **publishable research direction**
- or design a **novel scheduler architecture** around what you built
- or suggest **experiments + benchmarks (SWE-bench style)**

Just tell me 👍

📄 👍 💬 ↗️ ↺️ ... 🔗 Sources

I setup dynamo with sglang and with the ability to use genetic hints:

Harness sets hints in the request → Dynamo frontend parses next into routing hints → KV router uses them for queue ordering and worker selection → backends use them for priority scheduling and cache eviction.

What are some hot research areas in this ?

You've essentially built a **hint-driven control plane for LLM inference**. That's not just "cool infra" — it maps directly onto several *open research problems* people are actively trying to solve.

I'll focus on **where your exact architecture (genetic hints → routing → KV → scheduling → eviction) gives you leverage**.

🔥 1. Learned hint systems (from static → adaptive)

Right now your hints are *hand-authored*. The frontier is:

Can the system *learn* the optimal hints?

Research direction

- Train a policy that outputs:

- priority
- expected length
- reuse likelihood
- latency sensitivity

based on:

- prompt text
- historical traces
- user/session behavior

Techniques

- reinforcement learning (scheduler reward = latency/throughput)
 - bandits (explore vs exploit scheduling policies)
 - supervised (predict runtime, KV growth)
-

Why this is hot

Schedulers today are mostly heuristic. The shift is toward:

learned scheduling policies for LLM systems

Your advantage

You already have:

- a hint injection interface
- a closed loop system (hints → outcome)

👉 That's exactly what you need to train a policy.

🔥 2. Hint-conditioned KV cache management

You're already using hints for eviction. This is a huge area.

Core idea

KV cache is limited → must decide:

- what to keep
- where to store it

- when to move it
-

Open problems

- Predict **future reuse probability**
 - Trade off:
 - VRAM vs CPU vs disk
 - Avoid:
 - thrashing
 - recomputation
-

Research direction

Use hints as *priors* for cache value

Example:

- `reuse_prob = f(hint, prefix_similarity, session_state)`
 - `eviction_score = priority × reuse_prob / size`
-

Extensions

- **multi-tier KV systems**
 - **cache migration policies**
 - **cross-node KV sharing**
-

Why it's hot

Modern inference is often:

memory-bound, not compute-bound



3. Semantic routing (beyond load balancing)

You're already routing based on hints — this is cutting edge.

Traditional routing:

- round robin
- least loaded

New direction:

route based on *meaning of the request*

Research problems

- Route to node with:
 - matching KV prefix
 - best cache locality
 - specialized model state
-

With your system:

Hints can encode:

- "this belongs to session X"
- "this shares prefix with Y"
- "this is long-context"

👉 That enables:

cache-aware + semantic routing

Frontier idea

Build a:

KV-aware routing graph

Nodes are selected based on:

- cache overlap
 - hint similarity
 - predicted reuse
-



4. Agent-phase / workflow-aware scheduling

This is one of the biggest unsolved problems.

LLM workloads are not uniform:

- planning
- tool calls
- reasoning
- generation

Each phase has different needs.

Research direction

Use hints to encode:

- phase type
 - expected compute vs IO
 - urgency
-

Then schedule differently:

- planning → low latency, high priority
 - long generation → batchable, throughput optimized
 - tool wait → deprioritize
-

Why it matters

Most schedulers assume:

all requests are the same

They're not.

Your system

Already supports:

- injecting phase-level metadata

👉 That's rare and powerful.

🔥 5. Non-clairvoyant scheduling with hint augmentation

Classic problem:

scheduler doesn't know job size

LLMs make this worse:

- output length unknown
 - KV growth dynamic
-

Research direction

Hints provide:

partial clairvoyance

Examples:

- predicted token length
 - expected latency class
 - reuse likelihood
-

Open questions

- How accurate do hints need to be?
 - How robust is scheduler to bad hints?
 - Can system *correct* bad hints?
-

Big idea

"Hint-augmented non-clairvoyant scheduling"

6. Hint reliability & adversarial robustness

This is underexplored but critical.

If hints control:

- priority
- cache retention
- routing

👉 users can game the system

Research problems

- Detect:
 - dishonest hints
 - inflated priority
 - Design:
 - incentive-compatible systems
 - Enforce:
 - fairness constraints
-

Techniques

- trust scoring
 - hint auditing
 - anomaly detection
-

This becomes:

mechanism design for LLM infrastructure

7. Multi-objective scheduling (latency vs cost vs throughput)

Right now systems optimize one thing.

Reality:

- latency (p99)
 - throughput
 - GPU utilization
 - memory pressure
 - cost
-

Research direction

Use hints to express:

- SLA requirements
 - budget constraints
-

Then solve:

multi-objective optimization problem

Example

- high priority → low latency, expensive
 - low priority → batched, cheap
-

This becomes:

QoS-aware LLM infrastructure



8. Cross-request KV sharing & clustering

Huge opportunity.

Idea

Group requests by:

- prefix similarity
 - session
 - hint similarity
-

Then:

- co-schedule them
 - share KV cache
 - batch efficiently
-

Research angle

clustering + scheduling + cache reuse

With your system

Hints can explicitly signal:

- "same group"
 - "same prefix class"
-

9. Economic / market-based scheduling

This is emerging but not mature yet.

Idea

Treat resources as scarce:

- GPU time
 - KV memory
-

Use hints to encode:

- willingness to pay
- importance
- urgency

Then:

- allocate via auction / pricing
-

This becomes:

LLM inference markets

10. End-to-end co-design (model + scheduler)

Right now:

- model is fixed
- scheduler adapts

Future:

model and scheduler co-designed

Examples

- model emits:
 - expected length
 - uncertainty
 - scheduler uses that directly
-

With your system

Hints could come from:

- upstream model
 - not just harness
-

Where YOU are uniquely positioned

Your pipeline:

hints → routing → KV → scheduling → eviction



This is basically:

a programmable inference stack

Very few people have this end-to-end control.

If I were you (high-impact research ideas)

1. Learn a hint policy

- input: prompt + history
- output: hints
- optimize: latency + throughput

2. Build a KV value function

- predict:
 - reuse probability
 - future cost savings

3. Hint-aware router

- route to maximize:
 - cache reuse
 - locality

4. Robust hint system

- detect bad hints
- correct them automatically

5. Benchmark it

Create something like:

- "HintBench":
 - evaluate schedulers with/without hints
-

Big picture

You've moved from:

"scheduler decides everything"

to:

"requests *participate* in scheduling"

That shift is **massive**.

TL;DR (what's hottest for YOUR setup)

1. Learned hint generation (RL / bandits)
 2. Hint-aware KV cache management
 3. Semantic + cache-aware routing
 4. Workflow / agent-phase scheduling
 5. Robustness + incentive-compatible hint systems
-

If you want, I can help you turn this into:

- a **paper idea** (NeurIPS / MLSys level)
- or a **concrete experiment plan with metrics + baselines**



Give me research towards making the backend hardware (CPU, GPU, etc) work most efficiently with this direction. I want potential architectural innovations

I'll frame this specifically around your hint-driven Dynamo/SGLang stack and focus on architecture ideas that improve CPU/GPU/memory efficiency, not generic scheduling theory.

Thought for 5s >

Here are strong **hardware-efficiency research directions** for your hint-driven Dynamo + SGLang setup.

1. Hint-aware CPU/GPU co-scheduler

Use genetic hints to decide **where work should run and when**.

Potential innovation:

hint → predict phase → assign hardware path



Examples:

```
long_prefill=true      → send to compute-heavy GPU
decode_heavy=true      → send to bandwidth-optimized GPU
low_priority=true      → batch aggressively
latency_critical=true  → reserve fast lane
cache_hot=true         → route to worker holding KV
```



Why hot: Dynamo already emphasizes disaggregated prefill/decode, LLM-aware routing, async GPU transfers, and KV offloading across memory tiers. Your hint system can make these choices more intentional. NVIDIA Developer +1

2. Hint-driven prefill/decode disaggregation

Prefill and decode stress hardware differently:

```
prefill = compute-heavy
decode  = memory-bandwidth-heavy
```



Architectural idea:

```
Prefill GPU pool:
  high FLOPS GPUs
```



```
Decode GPU pool:
```

high memory-bandwidth GPUs

Hint router:
chooses ratio dynamically

Research angle:

Can hints predict the best prefill/decode split before execution?

This is especially important because KV transfer between prefill and decode workers is now a major bottleneck in disaggregated serving. [arXiv +1](#)

3. KV-cache “memory hierarchy controller”

Treat KV cache like an operating-system memory object.

HBM/GPU memory → CPU RAM → NVMe SSD → remote store



Your hints can control:

keep_in_gpu
spill_to_cpu
compress_before_spill
evict_early
pin_session_cache
prefetch_before_decode



Dynamo and LMCache are already moving toward tiered KV storage across GPU, CPU, SSD, and remote storage. [LMCache Blog +1](#)

Potential paper idea:

Hint-Guided Hierarchical KV Memory Management for LLM Serving

4. Cache-value function for hardware placement

Instead of simple LRU eviction, compute a value score:

KV_value =
priority
× predicted_reuse_probability



```

× recomputation_cost
÷ memory_size

```

Then decide:

```

high value → keep in GPU HBM
medium value → CPU RAM
low value → SSD or evict

```



SGLang's RadixAttention already uses prefix-tree KV reuse and cache-aware scheduling; your hints can become an extra signal inside that cache policy. llmsystem +1

5. Asynchronous KV prefetching

Use hints to fetch KV cache **before** the GPU needs it.

Architecture:

```

router sees request
↓
predict needed KV blocks
↓
CPU/NVMe starts prefetch
↓
GPU begins compute
↓
KV arrives just in time

```



Research question:

Can hint-driven prefetch hide CPU→GPU/NVMe→GPU transfer latency?

This directly targets the hardware bottleneck: data movement, not raw compute.

6. Genetic-hint compiler

Turn high-level hints into low-level hardware actions.

Example:

```
</> JSON
```



```
{  
  "latency": "critical",  
  "reuse": "high",  
  "phase": "agent_planning",  
  "expected_tokens": 256  
}
```

Compiles into:

```
route_to_worker_with_prefix_cache=true  
batch_size_limit=small  
pin_kv_in_hbm=true  
eviction_priority=protected  
prefill_chunk_size=small  
decode_priority=high
```



This could be a major architectural contribution:

A compiler layer between request semantics and hardware scheduling.

7. Heterogeneous GPU routing

Not all GPUs should do the same job.

Example architecture:

```
A10G / L4:  
  cheap low-priority decode  
  experiments  
  background jobs  
  
A100 / H100:  
  latency-critical prefill  
  long-context jobs  
  high-priority users
```



Your hints can route jobs based on expected hardware fit.

Research angle:

Hint-aware heterogeneous inference clusters.

This is powerful because mixed hardware clusters are becoming more common.

8. CPU as an active inference assistant

Don't treat CPU as just "host overhead."

Use CPU for:

```
hint parsing
request classification
prefix lookup
KV compression/decompression
prefetch planning
queue reordering
low-priority cache storage
speculative scheduling decisions
```



Innovation:

CPU prepares the future while GPU generates the present.

SGLang already emphasizes low-overhead CPU scheduling and CPU/GPU overlap; your hint system can make that overlap smarter. [GitHub](#)

9. Hint-aware KV compression

Compress KV differently based on hints.

```
high_priority + active session:
    keep full precision in GPU

medium priority:
    quantized KV in CPU RAM

low priority:
    aggressively compressed KV on SSD
```



KV compression is becoming hot because KV cache memory is one of the biggest long-context bottlenecks; recent work like TurboQuant targets large reductions in KV memory with minimal accuracy loss. [Tom's Hardware](#)

10. Hardware-feedback loop

The system should learn from actual hardware counters.

Collect:

GPU utilization
HBM usage
PCIe/NVLink transfer time
KV hit rate
CPU RAM pressure
NVMe read latency
TTFT
tokens/sec
p95/p99 latency



Then adjust hints automatically.

Architecture:

hints → scheduler decision → hardware outcome → hint policy update



This turns your system into a self-optimizing backend.

Best architectural innovations to pursue first

I'd rank them like this:

1. Hint-guided KV memory hierarchy
2. Hint-aware prefill/decode routing
3. Cache-value function for eviction/placement
4. Asynchronous KV prefetching
5. Heterogeneous GPU routing
6. Genetic-hint compiler



The strongest research title would be:

Hint-Guided Hardware-Aware Scheduling for Efficient LLM Inference

or more specifically:

Genetic Hint-Driven KV Cache Placement and Routing for Disaggregated LLM Serving

These are awesome research directions. How can I setup an environment that evaluates all these directions.

I'll give you an evaluation environment you can actually build incrementally: start with a single-node harness, then extend to multi-worker Dynamo/SGLang experiments for routing, KV placement, and prefill/decode studies.

Thought for 6s >

Set it up like a **mini research cluster + repeatable benchmark harness**.

1. Build the evaluation stack

```
Client harness
  ↓ sends prompts + genetic hints
Dynamo frontend
  ↓ parses hints
Hint router / KV router
  ↓ chooses worker + queue order
SGLang workers
  ↓ priority scheduling + KV policy
Metrics collector
  ↓
Experiment dashboard
```



Dynamo already supports SGLang integration, disaggregated serving, KV-aware routing, and request cancellation, so use that as the base instead of building a full serving stack yourself. [NVIDIA Docs](#)

2. Start with 3 deployment modes

Mode A — single GPU baseline

Purpose: isolate SGLang behavior.

```
1 GPU
Dynamo frontend
```



1 SGLang backend
RadixAttention enabled

Measure:

TTFT
tokens/sec
GPU utilization
KV hit rate
queue wait time
p95/p99 latency



SGLang is useful here because it already includes RadixAttention, prefix caching, continuous batching, chunked prefill, and prefill/decode disaggregation support.

[sgl-project.github...](#)

Mode B — multi-worker KV routing

Purpose: test worker selection.

Dynamo frontend
KV-aware router
2–4 SGLang workers



Compare:

least-loaded routing
round-robin routing
KV-aware routing
hint-aware KV routing



Dynamo's SGLang backend specifically supports KV-aware routing, which makes this the cleanest place to test your hint-based routing changes. [NVIDIA Docs](#)

Mode C — disaggregated prefill/decode

Purpose: test hardware specialization.

Prefill worker pool
Decode worker pool



KV transfer layer
Hint-aware routing between them

Dynamo supports separating prefill and decode into independent worker pools; the prefill engine computes KV, transfers it, then decode continues generation.

[docs.dynamo.nvi...](#)

For serious multi-node experiments, RDMA matters because Dynamo warns that disaggregated deployments can suffer severe degradation without RDMA.

[docs.dynamo.nvi...](#)

3. Create workloads that stress each research direction

Use synthetic workloads first.

Workload 1 — cache reuse

70% shared system prompt
30% unique user prompt
multi-turn conversation



Tests:

KV reuse
RadixAttention benefit
worker locality
hint-aware cache retention



SGLang's RadixAttention stores KV cache in a radix tree and reuses common prefixes, so this workload directly tests the core cache-reuse path. [LMSYS Org](#)

Workload 2 — mixed priority

```
</> JSON
{
  "priority": "high",
  "latency_sla_ms": 800,
  "cache_policy": "pin_if_reusable",
```



```
"expected_output_tokens": 128  
}
```

Mix:

10% urgent
30% normal
60% background



Tests:

priority queues
starvation
p99 latency
fairness
throughput loss



Workload 3 — long-context pressure

8k / 16k / 32k / 64k prompts
multi-turn sessions
limited GPU memory



Tests:

GPU HBM pressure
CPU offload
cache eviction
KV compression
prefetching



KV cache grows with user count and context length, which is why offloading to CPU RAM, SSD, or remote stores is now a major serving-system direction. AlBrix

Workload 4 — prefill/decode imbalance

short prompt + long generation
long prompt + short generation
long prompt + long generation



Tests:

prefill worker saturation
decode worker saturation
pool sizing
hint-aware disaggregation



4. Define your baselines

You need clean comparisons:

B0: vanilla Dynamo + SGLang
B1: KV-aware routing only
B2: priority scheduling only
B3: hint-aware routing only
B4: hint-aware routing + hint-aware eviction
B5: hint-aware routing + eviction + prefetch
B6: learned hint policy



Your main claim becomes:

Hints improve hardware efficiency by reducing wasted GPU work,
improving KV reuse, reducing transfer stalls, and lowering p99 latency.



5. Collect the right metrics

Minimum set:

Latency:
TTFT
inter-token latency
end-to-end latency
p50 / p95 / p99

Throughput:
requests/sec
tokens/sec
completed tokens/sec/GPU

GPU:
utilization



HBM used
memory bandwidth
GPU idle time

CPU:

CPU utilization
RAM used
tokenizer time
scheduler overhead

KV:

KV hit rate
KV eviction count
KV bytes moved GPU↔CPU
KV bytes moved worker↔worker
recomputation avoided

Fairness:

high-priority SLA hit rate
low-priority starvation rate

6. Add a hardware profiler layer

Use:

nvidia-smi dmon
DCGM exporter
Nsight Systems
Prometheus
Grafana
Linux perf
iostat
pidstat



The most important hardware counters:

GPU SM utilization
GPU memory utilization
PCIe / NVLink transfer volume
CPU RAM bandwidth
NVMe read/write throughput
network/RDMA throughput



7. Repository layout

Use something like:

```
hintllm-eval/
workloads/
  cache_reuse.py
  mixed_priority.py
  long_context.py
  prefill_decode_mix.py

hints/
  schemas/
    genetic_hint_schema.json
  policies/
    static_policy.py
    learned_policy.py

clients/
  openai_client.py
  async_loadgen.py

dynamo_patch/
  frontend_hint_parser/
  router_hint_policy/
  eviction_policy/

metrics/
  collector.py
  prometheus.yml
  grafana_dashboard.json

experiments/
  configs/
    baseline.yaml
    kv_routing.yaml
    hint_routing.yaml
    hint_eviction.yaml
    disagg_pd.yaml

analysis/
  plot_latency.py
  plot_kv_hits.py
  plot_gpu_efficiency.py
```



8. Your first experiment

Start simple:

Experiment 1:
Does hint-aware routing improve KV reuse and TTFT?



Setup:

2 SGLang workers
same model
same prompt stream
shared-prefix workload



Compare:

round-robin
least-loaded
KV-aware
hint-aware KV routing



Measure:

TTFT
KV hit rate
GPU utilization
prefill recomputation count
p95/p99 latency



Expected result:

Hint-aware routing should improve cache locality,
reduce prefill recomputation,
and improve TTFT under shared-prefix workloads.



9. Then expand to the big architecture experiments

After Experiment 1 works, add these:

Experiment 2:
Hint-aware KV eviction under memory pressure



Experiment 3:

Hint-aware CPU/GPU/NVMe KV placement

Experiment 4:

Hint-aware prefill/decode worker selection

Experiment 5:

Hint-aware asynchronous KV prefetching

Experiment 6:

Learned hint policy vs manually assigned hints

10. Strong paper/eval framing

Your environment should answer:

Can semantic request hints improve hardware efficiency
in distributed LLM inference systems?



Main metrics:

lower TTFT
lower p99 latency
higher tokens/sec/GPU
higher KV hit rate
lower GPU idle time
lower recomputation
better SLA satisfaction



Best project name:

HintBench: A Benchmark for Hint-Guided Hardware-Aware LLM Serving



or:

GeneServe: Genetic-Hint-Guided Scheduling for Efficient LLM Inference



     ...   Sources